CV 5주차

머신러닝 - 객체검출부터 특징점 매칭까지

1.1 템플릿 매칭

원본 영상에서 찾고자 하는 템플릿 이미지를 이동시키며 각 위치의 유사도를 계산하고, 가장 일치하는 영역을 검출하는 기법이다.

(1) TM_CCORR

$$R(x,y) = \sum_{x',y'} T(x',y') \cdot I(x+x',\, y+y')$$

- 템플릿과 영상 패치의 **곱의 합**
- 값이 클수록 두 영상이 유사

(2) TM_CCOEFF

$$R(x,y) = \sum_{x',y'} \big(T(x',y') - \bar{T}\big)\big(I(x+x',y+y') - \bar{I}\big)$$

- 평균을 뺀 뒤 상관성을 계산
- **밝기 변화 영향이 비교적 적음**
- 값이 클수록 잘 매칭됨

여기서

$$\bar{T} = \text{템플릿 평균}, \quad \bar{I} = \text{해당 영상 패치 평균}$$

(3) TM_SQDIFF

$$R(x,y) = \sum_{x',y'} \big(T(x',y') - I(x+x',\, y+y')\big)^2$$

- 템플릿과 영상 패치의 **차이 제곱합**
- 값이 **작을수록** 더 유사

1.1 템플릿 매칭

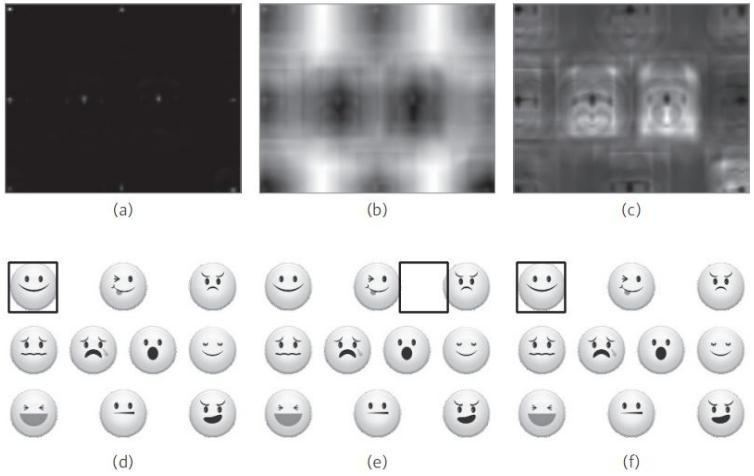
실험 결과 분석 (그림 3-2)

(a)~(c): 각 매칭 방법으로 계산한 유사도 맵

(d) TM_CCOEFF: 웃는 얼굴이 정확하게 검출됨

(e) TM_CCORR: 다른 위치가 선택되어 오검출 발생

(f) TM_SQDIFF: 웃는 얼굴이 정확하게 검출됨



[그림 3-2] 탬플릿 매칭에 의한 객체 검출
(a) TM_CCOEFF 유사도 (b) TM_CCORR 유사도 (c) TM_SQDIFF 유사도
(d) TM_CCOEFF 결과 (e) TM_CCORR 결과 (f) TM_SQDIFF 결과

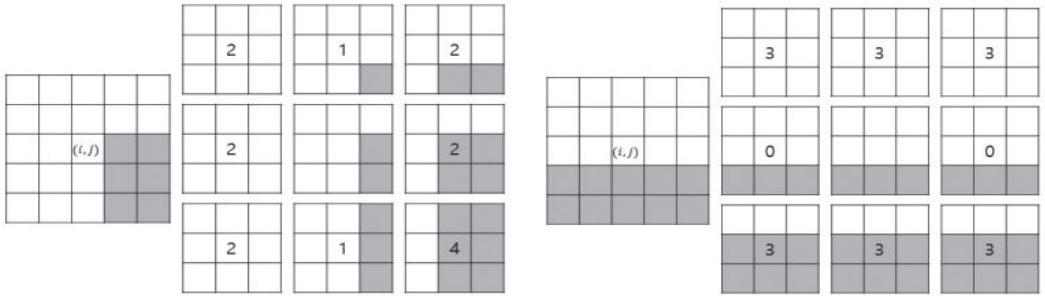
1.2 코너점 검출

코너점 검출은 영상의 특정 영역을 여러 방향으로 이동시키면서 픽셀의 밝기 변화량을 비교하여 코너점을 찾는 방법이다.

- 왼쪽(코너점): 상하좌우 어느 방향으로 이동하더라도 밝기 변화가 크게 발생한다.

- 오른쪽(직선): 직선과 평행한 방향으로 이동하면 밝기 변화가 거의 없지만, 수직 방향으로 이동하면 큰 변화가 발생한다.

- 핵심 원리: 코너점은 모든 방향에서 밝기 변화가 크고, 직선은 특정 방향에서만 밝기 변화가 크다.



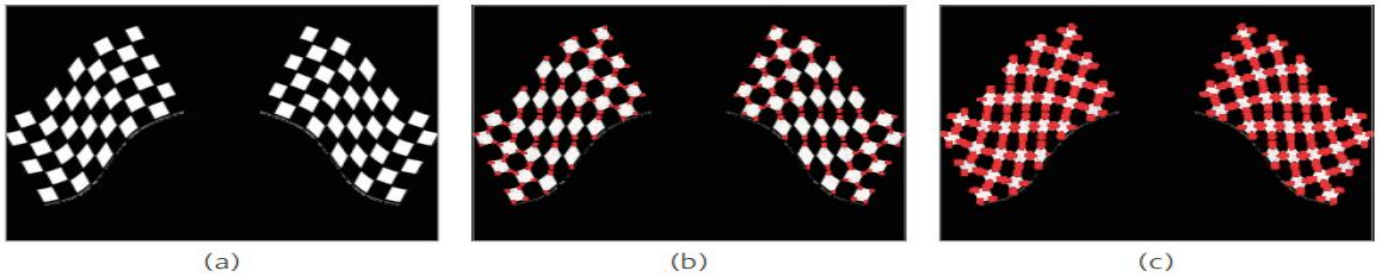
[그림 3-3] 코너점과 직선 주변일 경우의 자기 상관 값의 차이

1.2 코너점 검출

해리스 코너 검출은 영상에서 가로와 세로 방향의 밝기 변화량을 분석하여 코너점을 찾는 방법이다.

- (b) blockSize = 2: 작은 영역을 기준으로 밝기 변화를 계산하여 코너점을 검출한다. 비교적 세밀한 코너점 검출이 가능하다.

- (c) blockSize = 5: 더 넓은 영역을 기준으로 밝기 변화를 계산하여 코너점을 검출한다. 코너 응답이 나타나는 영역이 넓어져 더 많은 픽셀이 표시된다.



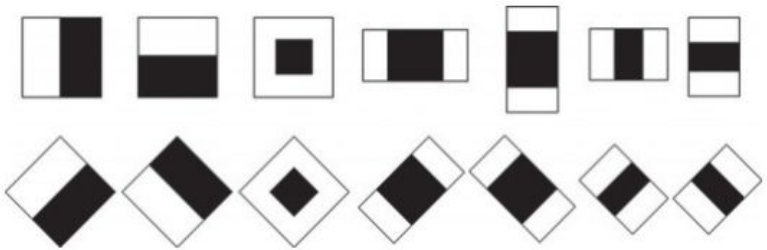
[그림 3-4] 해리스 코너 검출

(a) 입력 영상 (b)해리스 코너(blockSize=2) (c) 해리스 코너(blockSize=5)

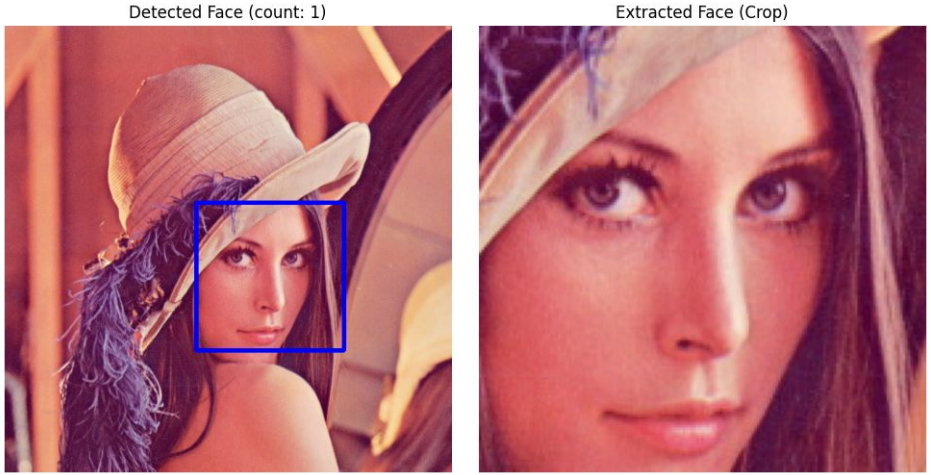
1.3 하라이크 필터

흰색과 검은색 영역의 픽셀 합 차이를 계산하는 명암 필터를 사용한다.

- 좌측 상단 누적합으로 필터가 켜져도 덧셈과 뺄셈 2회만으로 고속 연산이 가능하다.



하라이크 필터 예시



1.4 HOG 특징

Histogram of Oriented Gradients

- 영상의 지역적인 에지를 특징으로 사용한다.

1. 픽셀 단위로 그래디언트를 구하고, 8 x 8 단위 셀로 묶어 히스토그램을 만든다.
2. 이후 인접한 셀 4개를 묶어 블록을 만들고 1칸씩 오른쪽과 아래로 한 칸씩 이동한다.
3. 블록별 값을 한 줄로 나열하여 HOG 특징을 구할 수 있다.

1. 96 x 96 -> 12 x 12 개의 셀로 분할
2. 슬라이딩 윈도우 방식으로 11x11개의 블럭 생성 (블럭당 36 dim)
3. 11x11개의 블럭을 나열하여 feature vector 생성

11 x 11 x 36 = 4356 dim

```
img = cv2.cvtColor(cv2.imdecode(arr, cv2.IMREAD_COLOR), cv2.COLOR_BGR2RGB)
img_resized = cv2.resize(img, (96, 96))

features, hog_img = hog(
    img_resized,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    channel_axis=-1
)

fig, ax = plt.subplots(1, 2, figsize=(8, 4))
ax[0].imshow(img_resized)
ax[0].set_title('Resized Lenna (96x96)')
ax[0].axis('off')

ax[1].imshow(hog_img, cmap='gray')
ax[1].set_title(f'HOG Visualization (dim: {len(features)})')
ax[1].axis('off')

plt.tight_layout()
plt.show()
```

0.9s



1.5 LBP

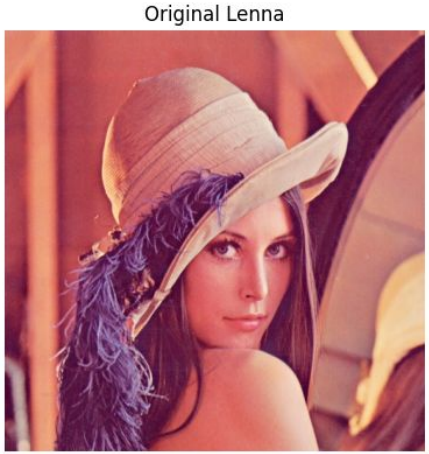
Local Binary Pattern

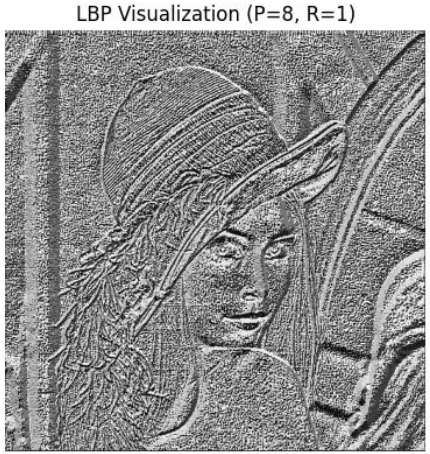
- 이웃한 픽셀들 간의 밝기 크기 관계를 이진 패턴으로 변환해 질감과 형상을 추출한다.

1. 3 × 3 윈도우에서 중앙 픽셀값을 임계치로 삼아 주변 8개 픽셀을 이진화(0 또는 1)한다.

2. 반시계 방향으로 나열한 8자리 이진수를 십진수(0~255)로 변환해 대표값으로 만든다.

3. 모든 픽셀에서 구한 LBP 값들을 히스토그램으로 나타내어 특징 벡터를 생성한다.





2.1 특징 및 기술자

Feature & Descriptor

- 영상에서 관심 영역을 식별하기 위한 핵심 위치와 주변 정보를 정의한다.

1. 특징(Feature) : 코너점, 에지 등 영상에서 두드러지게 구별되는 위치이다.

2. 기술자(Descriptor) : 특징점 주변의 밝기, 색상, 기울기 등을 수치화한 매칭 정보

예시로 HOG에서는 그래디언트를 Feature 로,
이를 셀, 블럭 단위 히스토그램으로 묶는 방식을 Descriptor로 볼 수 있다.

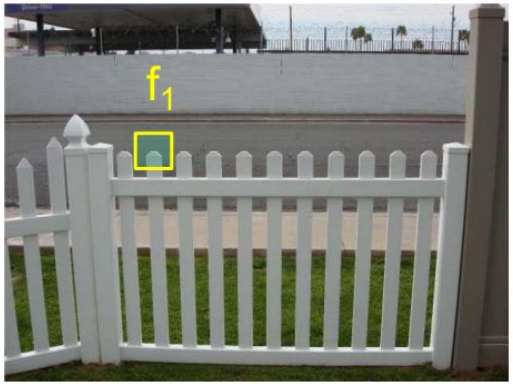
2.3 특징점 매칭

- 서로 다른 영상에서 추출한 특징점의 기술자를 비교하여 동일한 위치나 객체를 찾는 방법

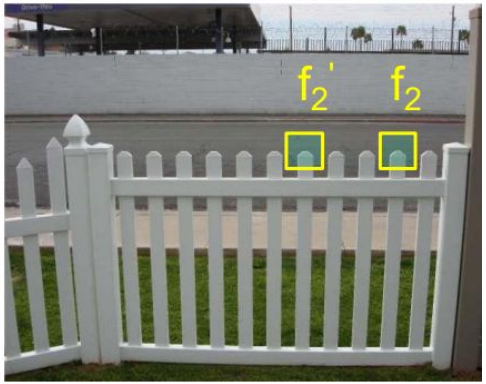
1. 단순 매칭의 한계 : 기술자 유사도만 비교하면 비슷한 패턴이 반복될 때 오매칭이 발생한다.

2. 관계 고려의 필요성 : 개별 특징점의 일치 여부뿐 아니라, 특징점 간의 위치·배열 관계 또한 중요하다.

3. L1 ↔ L2 매칭 : 유사한 객체가 여러 개 등장할 때, 어떤 객체끼리 올바르게 대응시킬지 고려해야 한다.



I_1



I_2

2.3 특징점 매칭

탐색 효율과 매칭 정확도를 높이기 위해 적합한 매칭 알고리즘과 오매칭 제거 기법을 사용한다.

1. Brute-Force Matcher
- 모든 후보 기술자를 일일이 전수 비교한다. 특징점 개수의 제곱에 비례하여 연산량이 증가한다.

2. FLANN Matcher
- 근사 최근접 이웃 탐색(KD-Tree, LSH)을 사용해 대용량 데이터에서도 빠르게 매칭한다.
- 전수 비교와 달리 근사 탐색을 사용하므로 가장 가까운 후보를 놓칠 수 있다.

3. 매칭 결과 개선
- Ratio Test : 최근접 후보와 두 번째 후보의 거리 비율을 비교하여 모호한 매칭을 제거한다.
- RANSAC : 무작위로 대응점을 선택하여 기하학적 변환을 추정하고, 이를 반복해 일관되지 않은 매칭을 제거한다.

실습 코드

https://colab.research.google.com/drive/1qqndtM_RhNd9irtKJvR9xcLiodvSJ9ry?usp=sharing